

OpenAI Agents SDK

LLM 앱의 진화

- 정적인 시스템에서 지능형 시스템으로 진화

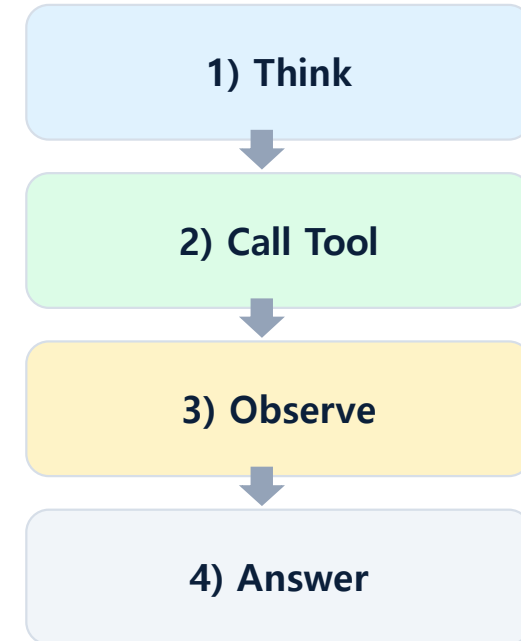
Chatbot → RAG App → RAG + MCP → Agentic AI

- LLM은 다음이 불가능
 - 실시간 데이터 접근
 - 파일 열기
 - 개인 문서 이해

Agents: ReAct 루프의 핵심

Agent 란 ?

- LLM을 핵심 엔진으로 사용하여 주어진 목표를 달성하기 위해 독립적으로 작업(추론, 판단, 실행, 피드백)을 수행하는 인공지능 시스템
- 주로 LLM(대규모 언어 모델)의 능력을 기반으로 동작하며, 사용자의 명령을 이해하고, 판단하며, 실행
- 즉, LLM은 두뇌, Agent는 이 두뇌를 활용해 행동을 실행하는 작업자 역할
- 활용 사례
 - 고객 지원 챗봇: 전자상거래 플랫폼에서 환불 요청을 처리하거나 배송 상태 확인
 - 연구 도우미: 과학 논문에서 특정 주제에 대한 핵심 내용 추출
 - 개인 비서: 이메일 초안을 작성하고, 미팅 일정 조율
 - 데이터 분석 에이전트: 주식 시장 데이터를 분석해 투자 전략 제안
 - 교육용 튜터: 수학 문제 풀이 과정을 단계별로 설명



OpenAI Agents SDK 핵심 구성 요소

- **모델 (Model)** : 고급 추론 능력과 에이전트 기능 내재 모델
- **에이전트 (Agents)** : LLM에 지시문과 도구를 장착한 실행 단위
- **핸드오프 (Handoffs)** : 특정 작업을 다른 에이전트에게 위임할 수 있도록 해 줌
- **가드레일 (Guardrails)** : 에이전트에게 전달되는 입력을 검증하는 장치
- **함수 기반 도구 (Function Tools)** : 에이전트가 호출할 수 있는 Python 함수 기반 도구
- **세션 (Session)** : 에이전트 실행 반복 과정(Agent Loop) 동안 필요한 작업 맥락을 유지해주는 영속적 메모리 레이어
- **추적 (Tracing)** : 에이전트 실행에 대한 내장 추적 기능

핸드오프 (Handoffs)

핸드오프 (Handoffs) 란?

- 에이전트가 다른 에이전트에게 작업을 위임하는 기능
- 각 에이전트가 서로 다른 전문 영역을 담당할 때 유용
- 예: 고객 지원 앱에서 **주문 조회**, **환불**, **FAQ** 등을 각각 전담하는 에이전트 구성

핵심 동작 원리

- Handoff는 LLM에게 **도구(Tool)**로 표현됨
- "Refund Agent"로의 핸드오프 → LLM이 **transfer_to_refund_agent** 도구 호출

핸드오프 (Handoffs)

Handoff 생성 방법

■ 방법 1: handoffs 파라미터에 Agent 객체 직접 전달

- handoffs 파라미터에 위임 대상 **Agent** 객체를 전달
- **handoff_description** 설정 → LLM이 언제 이 핸드오프를 선택할지 판단하는 힌트

■ 방법 2: handoff() 함수 사용

- 위임 대상 에이전트를 지정하면서 **세부 제어**가 가능
- 추가 설정(**override**)이나 **입력 필터링** 같은 고급 기능 활용 가능

가드레일 (Guardrails)

가드레일 (Guardrails) 이란?

- 에이전트와 병렬로 실행되며, 사용자 입력에 대해 검사와 유효성 검증을 수행
- 빠르고 저렴한 모델을 활용하여 입력이 악의적인 목적인지 먼저 검사
- 의심스러운 입력 감지 시 즉시 오류를 발생시켜 고비용 모델 실행을 차단
- 시간과 비용을 절약하는 입력 보호 장치

예시 시나리오

- 똑똑하지만 느리고 비용이 많이 드는 모델을 사용하는 에이전트를 악의적 사용자가 수학 숙제 대행 같은 목적에 맞지 않는 요청하는 경우
- 가드레일이 빠르고 저렴한 모델로 먼저 검사 → 감지 → 즉시 차단

구조화된 출력 (Structured Output)

- 모델이 특정한 출력 구조를 따르도록 지시 받는 것
- 모델의 응답이 데이터베이스 스키마나 다른 정해진 형식에 부합하도록 보장
→ 모델 출력의 일관성 유지
- 구조화된 출력 구현 방법 → 출력 구조를 스키마로 정의
- 스키마는 JSON과 같은 단순한 형식으로 표현
- Pydantic과 같은 도구를 사용하여 타입 힌트와 유효성 검사를 포함한 스키마 정의

```
from pydantic import BaseModel
from agents import Agent, Runner

# 캘린더 이벤트 데이터 구조 정의
class CalendarEvent(BaseModel):
    name: str          # 이벤트 이름
    date: str          # 이벤트 날짜
    participants: list[str] # 참가자 목록

# 에이전트 정의: 텍스트에서 캘린더 이벤트 정보를 추출
agent = Agent(
    name="캘린더 추출기", # 에이전트 이름
    instructions="텍스트에서 캘린더 이벤트를 추출하세요. "
                "이벤트 이름, 날짜, 참가자 정보를 구조화된 데이터로 반환하세요.",
    output_type=CalendarEvent, # 반환 데이터 형식
    model="gpt-5-mini",       # 사용할 LLM 모델
)
```


Tools & Tool Calling

LLM의 함수 호출 방법

LLM은 함수 호출(Function Calling) 기능을 통해 외부 도구 및 API와 안정적으로 통합될 수 있습니다.

JSON 기반 함수 호출:

모델은 함수 시그니처를 따르는 JSON 객체로 응답합니다.

개발자는 함수의 입력 파라미터를 JSON 스키마로 설명합니다.

자동 함수 호출 결정:

모델은 입력에 따라 적절한 함수를 호출할 시점을 감지합니다.

사용 사례:

외부 도구 호출: API, 플러그인 등과 연결해 작업 수행.

예: `get_current_weather(location: string, unit: 'celsius' | 'fahrenheit')`

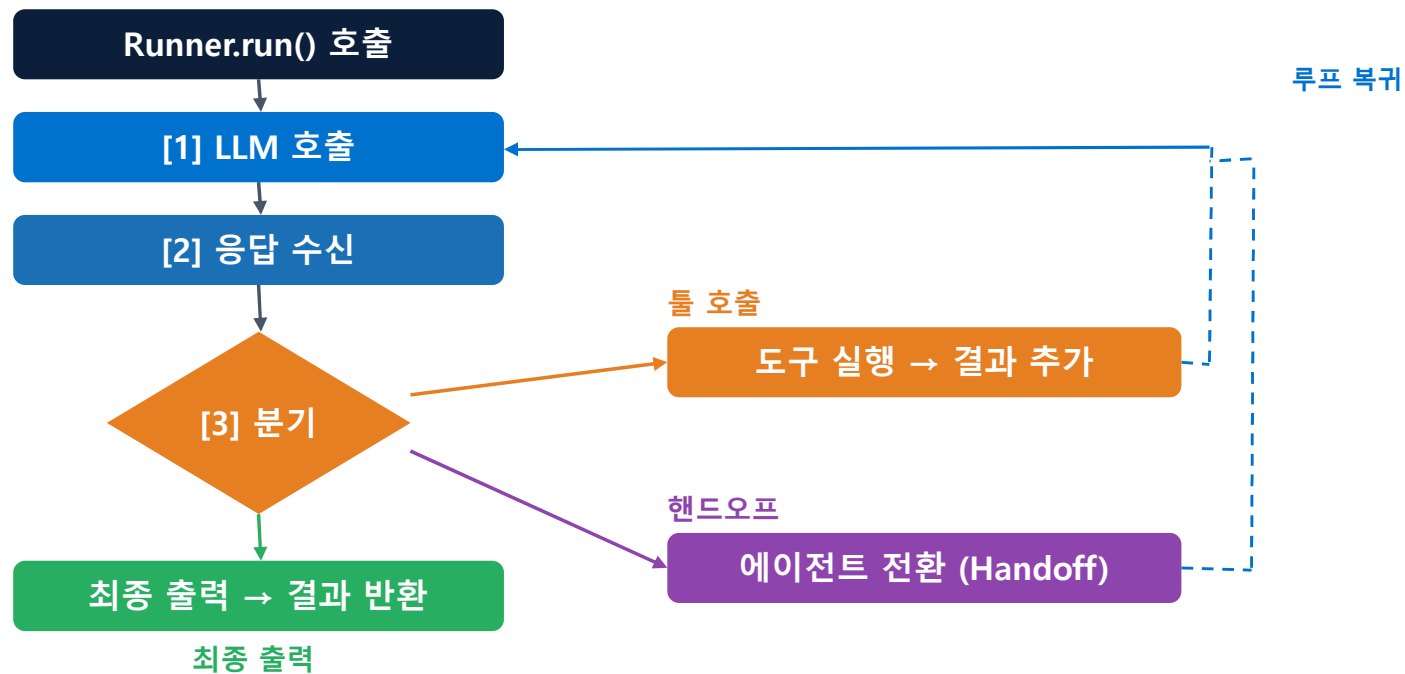
```
@function_tool
def multiply(x: float, y: float) -> float:
    """
    x 와 y 를 곱한다.
    """
    print("** multiply 함수 실행 **", x, y)
    return x * y

@function_tool
def get_weather(위도: float, 경도: float) -> str:
    print(위도, 경도)
    print(f"Weather 함수 실행 - 도시: {위도, 경도}")
    latitude=위도
    longitude=경도
    response = requests.get(f"https://api.open-meteo.com/v1/forecast?latitude={latitude}&longitude={longitude}&current_weather=true")
    data = response.json()
    return data['current']['temperature_2m']

agent = Agent(
    name="Assistant",
    instructions=(
        "당신은 유용한 도우미 입니다. 가능하면 제공된 도구를 사용하세요."
        "당신은 기상 전문가 입니다. 도시의 경도와 위도를 주면 날씨를 알려줍니다."
        "사용자 질의에 답하기 위해 당신의 지식에만 너무 의존하지 말고 도구를 사용하세요."
    ),
    model="gpt-5-nano",
    tools=[multiply, get_weather]
)
```

Agent Loop 흐름

에이전트 루프 동작 순서



툴 호출/핸드오프 → [1]로 돌아가 LLM 재호출, 최종 출력 시 루프 종료

■ Start ■ [1] LLM ■ [2] 응답 ■ 툴 호출 ■ 핸드오프 ■ 완료

output_type 3가지 케이스

output_type에 따른 최종 출력 결정

Case 1: output_type 미지정

- LLM이 자연어 문자열로 직접 응답 → result.final_output = 문자열

Case 2: output_type 지정 (구조화된 출력)

- Pydantic 모델 지정 → LLM 응답이 모델에 맞으면 구조화된 객체 반환

```
class WeatherResult(BaseModel):
```

```
    city: str; temperature: float; condition: str
```

```
agent = Agent(output_type=WeatherResult, tools=[get_weather])
```

Case 3: 핸드오프 + output_type

- 분류(Triage) 에이전트 → 전문 에이전트로 핸드오프
- 전문 에이전트에 output_type 설정 → 구조화된 결과 반환

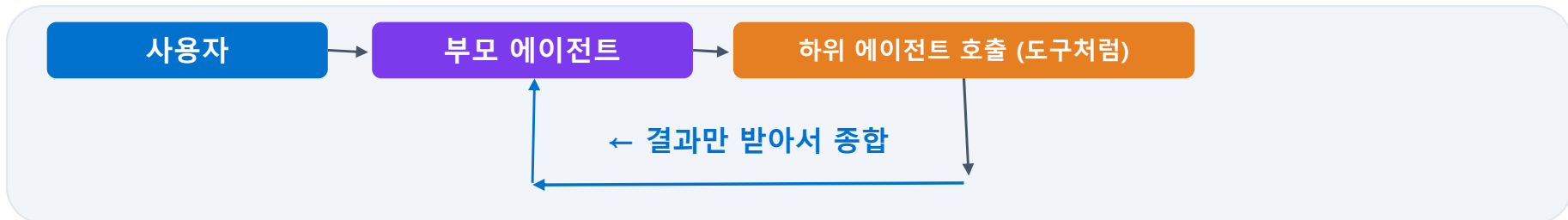
agent.as_tool() vs Handoff

Handoff - 제어권 완전 이전



예: 고객센터에서 환불 담당자에게 연결해주는 것

agent.as_tool() - 부모가 제어권 유지



예: 부모가 요약도 시키고 번역도 시킨 뒤 결과를 모아서 사용자에게 답변

01. OpenAI Agent (Agents Overview)

OpenAI 플랫폼을 사용하여 사용자를 대신해 행동(예: 컴퓨터 제어 등)을 수행할 수 있는 에이전트(Agent)를 구축할 수 있습니다. Python용 Agent SDK를 사용하면 백엔드에서 이러한 에이전트의 오케스트레이션(조율) 로직을 만들 수 있습니다.

OpenAI Agents SDK는 에이전트 기반 AI 애플리케이션을 개발할 수 있게 해주는 도구입니다.

핵심 구성 요소

- 에이전트 (Agents):
LLM에 지시문(instructions) 과 도구(tools) 를 장착한 실행 단위
- 핸드오프 (Handoffs):
특정 작업을 다른 에이전트에게 위임할 수 있도록 해줌
- 가드레일 (Guardrails):
에이전트에게 전달되는 입력을 검증하는 장치

Runner 3종 비교

Runner 실행 방식 비교표

Runner.run() | await (비동기) | 에이전트 루프 자동 실행, 도구/핸드오프 지원 | Jupyter, FastAPI

Runner.run_sync() | 동기 호출 | 내부적으로 asyncio.run() 사용 | 스크립트, 테스트

Runner.run_streamed() | await (스트리밍) | 토큰 단위 실시간 출력 | 실시간 채팅 UI

코드 예시

비동기 실행

```
result = await Runner.run(starting_agent=agent, input="질문")
```

동기 실행

```
result = Runner.run_sync(agent, "질문")
```

동기/비동기 실행 비교

SYNC 동기 실행 — 순차적, 블로킹



● 동기 (run_sync)

- 작업이 끝날 때까지 다음 작업 시작 불가
- 코드가 단순하고 직관적
- 이벤트 루프 불필요
- 일반 .py 스크립트에서 사용

ASync 비동기 실행 — 동시, 논블로킹



● 비동기 (run)

- I/O 대기 중 다른 작업 처리 가능
- async/await 문법 필요
- 이벤트 루프 위에서 동작
- Jupyter / 웹앱 / 서버에서 사용

02. Agent와 Runner

Runner는 에이전트 루프를 돌리는 실행기입니다.

- 동기: `Runner.run_sync(agent, input)` — 스크립트/테스트에 적합
- 비동기: `await Runner.run(agent, input)` — Jupyter/비동기 앱에 적합

에이전트 루프 동작:

1. LLM 호출 (에이전트 설정 + 메시지 히스토리)
2. 응답에 툴 호출이 있으면 실행 후 결과를 메시지에 추가하고 1번으로
3. 핸드오프가 있으면 대상 에이전트로 바꾸고 1번으로
4. **final_output**이 나오면 루프 종료 후 반환
5. `max_turns` 로 최대 턴 수 제한 가능

비동기 핸들러 - error_handlers

error_handlers 파라미터로 예외 처리

- **문제** : max_turns 초과 시 프로그램 크래시
- **해결** : error_handlers 파라미터로 폴백 메시지 반환

```
async def on_max_turns(ctx) -> RunErrorHandlerResult:  
    return RunErrorHandlerResult(  
        final_output="처리 한도를 초과했습니다...",   
        include_in_history=False)   
      
result = await Runner.run(agent, "...",  
    max_turns=3,  
    error_handlers={"max_turns": on_max_turns})
```

→ Fallback 메시지

include_in_history 옵션

- **True** : 폴백 메시지가 대화 기록에 저장됨
- **False** : 폴백 메시지가 사용자에게만 표시, 대화 기록에 남지 않음 → 권장

03. Error Handling

에이전트 루프에서 `max_turns`를 초과하면 기본적으로 `MaxTurnsExceeded` 예외가 발생하여 프로그램이 중단됩니다. `error_handlers`를 사용하면 예외 대신 제어된 최종 메시지를 반환하도록 할 수 있습니다.

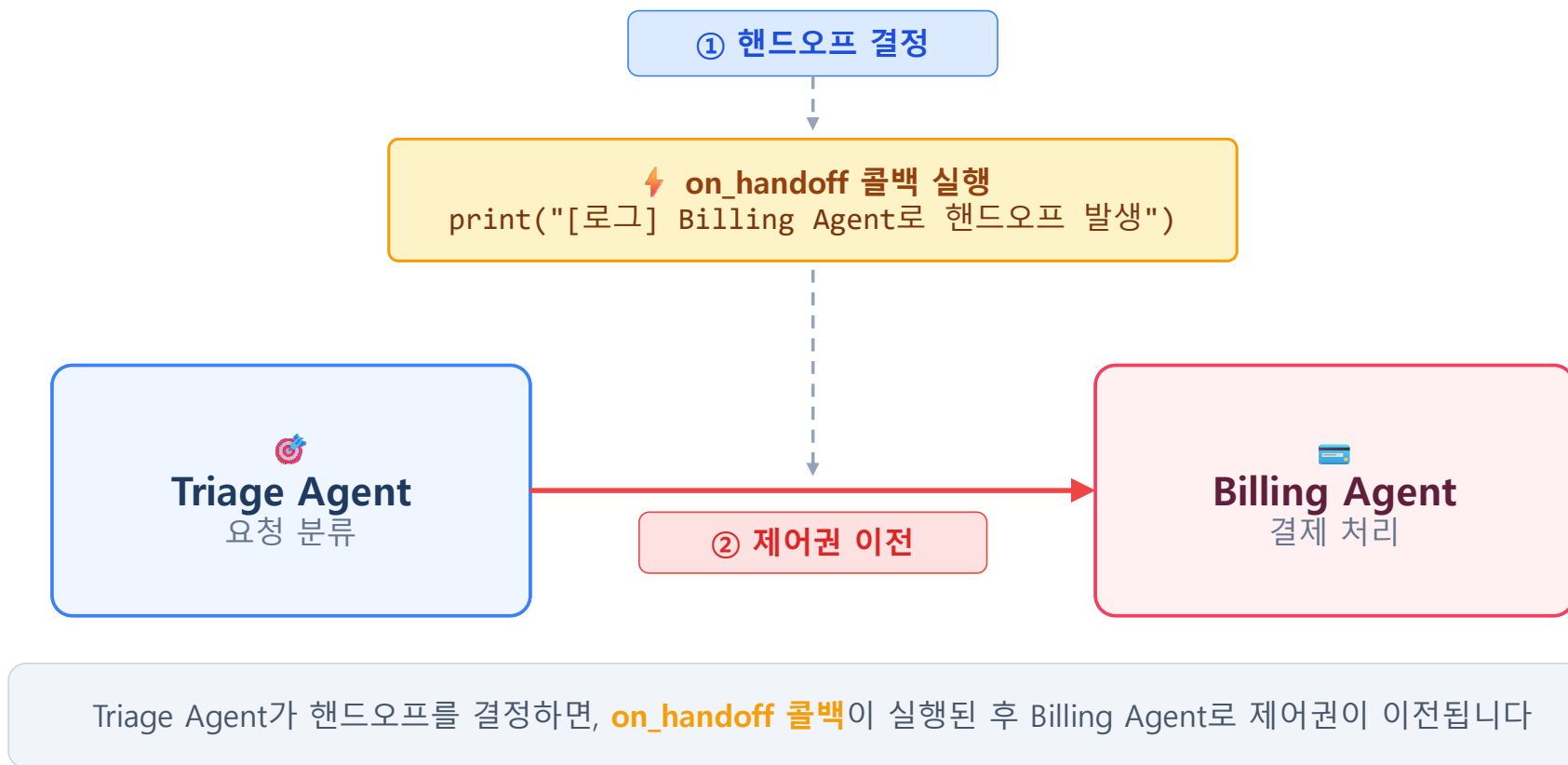
핵심 개념

- `error_handlers={"max_turns": handler}`: `Runner.run()` 호출 시 전달하여, 턴 초과 시 핸들러 함수가 실행되도록 설정
- `RunErrorHandlerResult`: 핸들러가 반환하는 결과 객체로, `final_output` (폴백 메시지)과 `include_in_history` 옵션을 포함
- `include_in_history=False`: 폴백 메시지를 대화 기록에 남기지 않음 (사용자 안내 전용)

학습 내용

1. `max_turns` 초과 시 `error_handlers` 로 예외 없이 안전하게 처리하기
2. 비동기(`async def`) / 동기(`def`) 핸들러 작성법
3. `include_in_history` 옵션의 역할과 활용

핸드오프 발생 시 콜백 실행



on_handoff 콜백

핸드오프 발생 시 콜백 실행

- 로깅, 알림, 상태 업데이트 등에 활용

```
def on_billing_handoff(ctx):  
    print("[로그] Billing Agent로 핸드오프 발생")  
  
    triage_agent = Agent(  
        handoffs=[  
            handoff(billing_agent, on_handoff=on_billing_handoff)])
```

핸드오프 후 응답 에이전트 확인

```
result = await Runner.run(triage_agent, "결제 오류가 있어요")  
print(result.last_agent.name) # "결제 에이전트"
```

대화 기록 필터링: 다음 에이전트에 전달되는 히스토리 제어

방법 1: 모든 도구 호출/결과 메시지 제거

```
handoff(faq_agent, input_filter=handoff_filters.remove_all_tools)
```

방법 2: 이전 대화를 하나의 어시스턴트 메시지로 압축

```
handoff(summary_agent, input_filter=handoff_filters.nest_handoff_history)
```

input_filter 비교

- **remove_all_tools** : 도구 호출/결과 메시지 삭제 → FAQ 에이전트에 깔끔한 대화만 전달
- **nest_handoff_history** : 전체 대화를 1개 메시지로 압축 → 요약 에이전트에 컨텍스트 전달

RECOMMENDED_PROMPT_PREFIX

핸드오프 프롬프트 삽입

- **문제** : LLM이 핸드오프 가능 여부를 모를 수 있음
- **해결** : 핸드오프 지시문을 프롬프트에 자동 삽입

방법 1: 프롬프트 앞에 직접 추가

```
agent = Agent(  
    instructions=RECOMMENDED_PROMPT_PREFIX + "당신은 고객 지원 에이전트입니다.",  
    handoffs=[refund_agent, order_agent])
```

방법 2: 함수로 래핑

```
agent = Agent(  
    instructions=prompt_with_handoff_instructions("..."),  
    handoffs=[refund_agent, order_agent])
```

04. Handoffs (핸드오프 심화)

핸드오프(**Handoff**) 는 에이전트가 특정 작업을 다른 에이전트에게 위임하는 기능입니다.

핸드오프는 LLM에게 도구(**tool**) 로 표현됩니다.

예) Refund Agent 로의 핸드오프 → LLM 도구 이름: `transfer_to_refund_agent`

방식	예시	특징
Agent 직접 전달	<code>handoffs=[billing_agent]</code>	간단, 기본 동작
<code>handoff()</code> 함수	<code>handoffs=[handoff(billing_agent, on_handoff=...)]</code>	콜백, 이름 재정의, 입력 필터 등 고급 옵션

`handoff()` 함수의 주요 파라미터:

파라미터	설명
<code>agent</code>	위임할 대상 에이전트
<code>tool_name_override</code>	LLM에게 노출되는 도구 이름 재정의 (기본: <code>transfer_to_<agent_name></code>)
<code>tool_description_override</code>	도구 설명 재정의
<code>on_handoff</code>	핸드오프 호출 시 실행되는 콜백 함수
<code>input_type</code>	LLM이 핸드오프 시 함께 전달할 데이터 타입 (Pydantic 모델)
<code>input_filter</code>	다음 에이전트에게 전달되는 대화 기록 필터링 함수
<code>is_enabled</code>	핸드오프 활성화 여부 (bool 또는 런타임 함수)

SDK 제공 Session 8종

- **SQLiteSession** : 내장, 인메모리/파일 (개발/프로토타입)
- **AsyncSQLiteSession** : 비동기 전용 SQLite
- **AdvancedSQLiteSession** : 브랜치, 토큰 분석 (고급)
- **SQLAlchemySession** : PostgreSQL, MySQL (프로덕션)
- **RedisSession** : Redis 분산 저장 (대규모 서비스)
- **EncryptedSession** : 암호화 래퍼 (보안 필수)
- **OpenAIConversationsSession** : OpenAI 서버 저장
- **CustomSession** : 사용자 정의 (Django, MongoDB 등)

05. Sessions

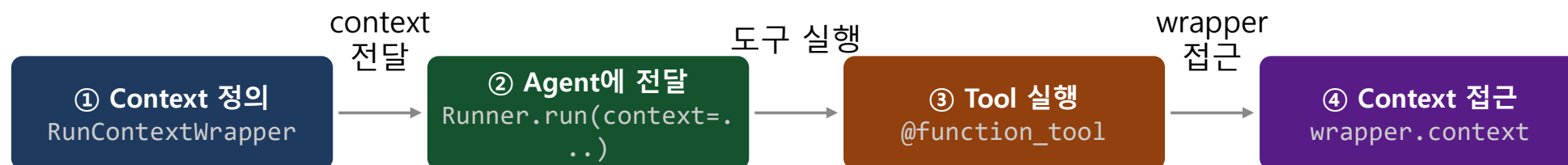
Session은 여러 번의 `Runner.run` 호출에 걸쳐 대화 히스토리를 자동으로 유지하는 기능입니다.

- 세션을 쓰지 않으면 매 턴마다 이전 메시지 목록을 직접 구성해 넘겨야 합니다.
- `SQLiteSession(session_id)` 를 사용하면 SDK가 대화를 DB에 저장/불러와서, 같은 `session` 으로 실행할 때마다 맥락이 이어집니다.

활용: 채팅봇, 고객지원 에이전트 등 다회차 대화.

기본 컨텍스트 - RunContextWrapper

RunContextWrapper 의존성 주입 흐름



Context Management 핵심 개념

- RunContextWrapper: 의존성 주입 패턴으로 Agent/Tool 간 상태(DB, API 클라이언트 등) 공유
- Agent 생성 시 context 타입 지정 → Runner.run()에서 context 인스턴스 전달
- Tool 함수에서 wrapper.context로 접근 → 타입 안전한 의존성 사용
- 장점: 글로벌 변수 없이 구조적이고 안전한 상태 관리 가능

핵심: RunContextWrapper로 Agent/Tool 간 상태 공유를 타입 안전하게 구현

06. Context Management (컨텍스트 관리) ¶

에이전트 실행 중 사용자 정보(user id, 권한 등), DB 연결 객체, API 키, 설정값 등을 도구 함수에 전달해야 할 때가 있습니다. `RunContextWrapper` 를 사용하면 이러한 데이터를 의존성 주입(Dependency Injection) 방식으로 전달할 수 있습니다.

핵심 개념

개념	설명
Local Context	도구 함수에서 접근 가능한 로컬 데이터 (LLM에 전송되지 않음)
RunContextWrapper[T]	컨텍스트를 감싸는 래퍼 클래스
ToolContext[T]	RunContextWrapper + 도구 메타데이터 (도구 이름, 호출 ID 등)

중요: 컨텍스트는 LLM에 전송되지 않습니다!

컨텍스트 객체는 로컬에서만 사용됩니다. 보안이 필요한 정보(API 키, DB 연결 등)를 안전하게 도구에 전달할 수 있습니다.

Output 가드레일

Output 가드레일 검증 흐름



정규식 가드레일 (RegexGuardrail)

- 정규표현식 기반 응답 필터링
- 패턴 매칭으로 금지어/민감 정보 탐지
- 빠른 실행 속도, 규칙 기반 검증

LLM 가드레일 (LLMGuardrail)

- LLM 기반 의미론적 응답 평가
- 문맥 이해 기반 위험도 판정
- 유연한 판단, 복잡한 규칙 처리 가능

tripwire 판정: 위반 시 처리

- tripwire_triggered=True 시 중단
- 위반 내용 로깅 및 사용자 알림
- 안전한 기본 응답으로 대체 출력

핵심: Output 가드레일은 Agent 응답의 안전성을 보장하는 최후 방어선

07. Output Guardrail (출력 가드레일)

01_Agents_Overview에서 `@input_guardrail` 로 입력을 검증하는 방법을 배웠습니다. 이번에는 `@output_guardrail` 로 에이전트의 출력을 검증하는 방법을 알아봅니다.

Input vs Output Guardrail 비교

구분	Input Guardrail	Output Guardrail
검증 시점	에이전트 실행 전	에이전트 실행 후
검증 대상	사용자 입력	에이전트 출력
데코레이터	<code>@input_guardrail</code>	<code>@output_guardrail</code>
예외	<code>InputGuardrailTripwireTriggered</code>	<code>OutputGuardrailTripwireTriggered</code>
실행 모드	Parallel(병렬) 또는 Blocking(차단)	항상 Sequential(순차)

Tripwire (트립와이어)란?

가드레일이 문제를 감지했을 때 발생시키는 즉시 중단 신호입니다. `tripwire_triggered=True` 로 설정하면 즉시 예외가 발생하고 실행이 중단됩니다.

다양한 시각화 - draw_graph()

draw_graph() 사용법

단일 에이전트 + 도구

```
from agents.extensions.visualization import draw_graph
draw_graph(weather_agent) # Jupyter 인라인 표시
```

멀티에이전트 (핸드오프)

```
triage = Agent(name="분류", handoffs=[ko_agent, en_agent])
draw_graph(triage)
```

파일 저장 & 브라우저

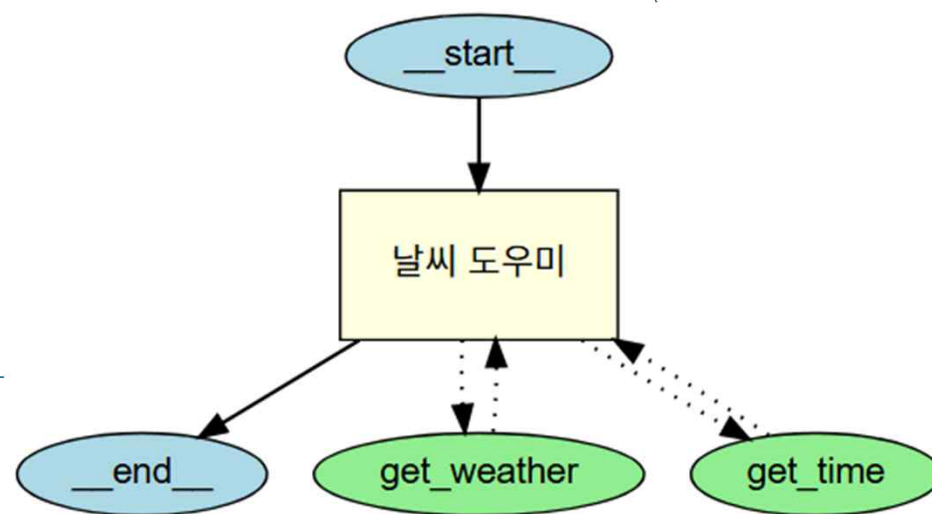
```
draw_graph(agent, filename="graph") # PDF 저장
draw_graph(agent).view()           # 뷰어에서 열기
```

08. Agent Visualization (에이전트 시각화)

OpenAI Agent SDK는 **Graphviz**를 사용하여 에이전트 구조를 시각적으로 표현하는 기능을 제공합니다.

시각화가 보여주는 것:

- 노란색 박스: 에이전트 (Agent)
- 녹색 타원: 도구 (Tool)
- 회색 박스: MCP 서버
- 실선 화살표: 핸드오프 연결
- 점선 화살표: 도구 연결
- 대시선 화살표: MCP 서버 연결



커스텀 트레이스 & 그룹핑

여러 실행을 하나로 그룹핑

커스텀 트레이스: 여러 실행을 하나로

```
from agents import trace
with trace("요약 및 평가 워크플로우"):
    summary = await Runner.run(summarizer, text)
    eval = await Runner.run(critic, summary.final_output)
```



group_id로 세션별 트레이스 관리

```
with trace("고객 요청 처리", group_id="session-abc123"):
    result = await Runner.run(support_agent, "...")
# 같은 group_id → 대시보드에서 묶어서 조회
```



9. Tracing (트레이싱)

에이전트가 어떤 단계를 거쳐 응답을 생성했는지 추적(Trace) 할 수 있습니다. 트레이싱은 디버깅, 모니터링, 성능 분석에 필수적인 기능입니다.

핵심 개념

개념	설명
Trace	하나의 워크플로우 전체를 나타내는 단위
Span	Trace 내의 개별 작업 단위 (LLM 호출, 도구 실행 등)
자동 트레이싱	SDK가 자동으로 에이전트 실행, LLM 호출, 도구 호출을 추적
Custom Span	개발자가 직접 정의하는 추적 단위

자동으로 추적되는 항목

- `Runner.run()` 전체 실행 → **Trace**
- 에이전트 실행 → **Agent Span**
- LLM 호출 → **Generation Span**
- 도구 호출 → **Function Span**
- 가드레일 → **Guardrail Span**
- 핸드오프 → **Handoff Span**

RAG 패턴 비교 + 실행 흐름

RAG 핵심 실행 흐름

- 1. 질의 → 2. 검색(Retrieve) → 3. 컨텍스트 구성
- 4. 프롬프트 + 문서 → 5. LLM 생성 → 6. 응답

Naive RAG vs Advanced RAG

- Naive: 단순 유사도 검색 → 생성
- Advanced: 쿼리 변환 + 재랭킹 + 필터링
- Agentic RAG: 에이전트가 검색 전략 결정

RAG 패턴 비교

Naive RAG

- 구현 간단, 빠른 프로토타입
- 한계: 노이즈 문서 포함 가능

Agentic RAG

- 에이전트가 검색 여부 판단
- 다중 소스 검색 가능
- 자기 평가로 품질 검증

Vector Store 연동 - RAG

1 파일 준비 & 업로드

파일 준비

PDF, TXT, DOCX 등 원본 문서 준비



Files API 업로드

client.files.create()로 파일 업로드



file_id 획득

업로드 완료 후 고유 file_id 반환

file_id 전달

2 Vector Store 구성

Vector Store 생성

자동 청킹 · 임베딩 · 인덱싱



파일 연결

file_id를 Vector Store에 연결



검색 준비 완료

File Search 도구로 검색 가능

원본 파일 → Files API 업로드 → file_id 발급 → Vector Store 생성 → 파일 연결 → 검색 준비 완료

Vector Store 연동 - RAG

파일 업로드 → 벡터 DB 자동 구성

Vector Store 생성

```
vs = client.vector_stores.create(name="제품 문서")
```

파일 업로드

```
file = client.files.create(  
    file=open("products.pdf", "rb"),  
    purpose="assistants")  
  
client.vector_stores.files.create(  
    vector_store_id=vs.id, file_id=file.id)
```

에이전트에 file_search 도구 연결

```
tool = FileSearchTool(vector_store_ids=[vs.id])
```

스트리밍 챗봇 코드

토큰 단위 실시간 출력

```
result = Runner.run_streamed(agent, input=messages)

full_response = ""
async for event in result.stream_events():
    if event.type == "raw_response_event" and
        isinstance(event.data, ResponseTextDeltaEvent):
        delta = event.data.delta or ""
        print(delta, end="", flush=True) # 실시간 토큰 출력
        full_response += delta
```

10. Chatbot RAG Agent 구현

OpenAI Agents SDK로 채팅 에이전트를 단계별로 확장합니다. 도구 없는 단순 챗봇에서 시작해 날씨/웹검색/RAG 도구를 하나씩 붙여가며 완성합니다.

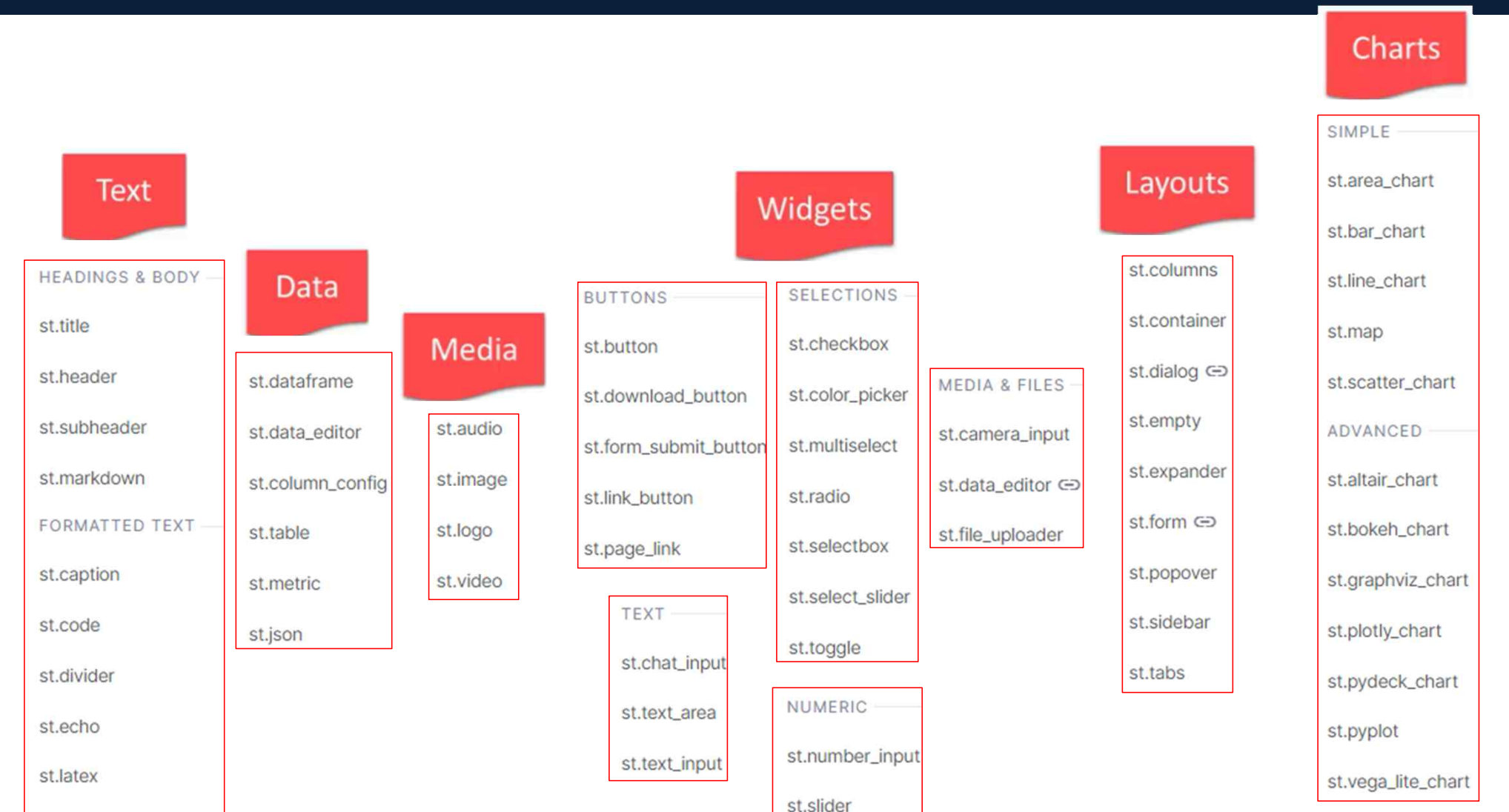
- | | |
|-----------|----------------------------------|
| 1. 기본 대화 | Agent + Runner.run() |
| ↓ | |
| 2. 스트리밍 | Runner.run_streamed() + 토큰 단위 출력 |
| ↓ | |
| 3. 도구 추가 | + get_weather + WebSearchTool |
| ↓ | |
| 4. RAG 추가 | + FileSearchTool (Vector Store) |
| ↓ | |
| 5. 웹 UI | → Streamlit app.py |

단계	내용	핵심 API / 도구
1. 비동기 방식 호출	도구 없이 Agent + Runner로 대화 구현 (메시지 유지)	<code>await Runner.run()</code>
2. Streaming 방식 호출	동일 에이전트를 토큰 단위 실시간 출력으로 실행	<code>Runner.run_streamed()</code> , <code>ResponseTextDeltaEvent</code>
3. 도구를 추가한 Chatbot	날씨 조회 + 웹 검색 도구를 에이전트에 연결	<code>@function_tool</code> <code>get_weather</code> , <code>WebSearchTool()</code>
4. RAG 도구 추가	OpenAI Vector Store에 FAQ/KB 업로드 후 검색 도구 연결	<code>FileSearchTool(vector_store_ids=[...])</code>
5. 웹 UI 연결	동일 에이전트를 Streamlit 웹 앱으로 구현한 예제 안내	<code>10_chat-app-rag/app.py</code>

What is Streamlit ?

- Machine Learning 과 Data Science를 위한 open-source application framework
- HTML, CSS, JavaScript와 같은 웹 기술을 알 필요 없이 파이썬으로 모든 것 처리
- 시각화 라이브러리와 쉽게 통합되어 다양한 데이터 시각화 가능
- Streamlit의 위젯을 사용하여 대화형 그래프와 대시보드를 만들 수 있다.
- 데이터가 변경될 때마다 실시간으로 애플리케이션을 업데이트
- 간단한 캐싱 메커니즘을 통해 빠른 성능을 유지
- 로컬에서 개발한 애플리케이션을 클라우드에서 손쉽게 배포하고 공유

Overview of Streamlit API



Text Display

```
app_01_text_display.py > ...
1  import streamlit as st
2
3  def main():
4      st.title("이것은 title입니다.")
5      st.header("이것은 header 입니다.")
6      st.subheader("이것은 subheader 입니다.")
7
8      st.text("Hello World Streamlit")
9      name = "김길동"
10     st.text("{}님, 안녕하세요. 반갑습니다.".format(name))
11
12     st.markdown("이것은  $\frac{1}{x}$ 입니다.")
13
14
15 if __name__ == '__main__':
16     main()
```



이것은 title입니다.

이것은 header 입니다.

이것은 subheader 입니다.

Hello World Streamlit

김길동님, 안녕하세요. 반갑습니다.

이것은 $\frac{1}{x}$ 입니다.

st.write – 만능 function

```
def main():  
    st.text(2+3)  
    st.write(2+3)  
  
    st.markdown("## 이것은 title입니다.")  
    st.write("## 이것은 title입니다.")  
    st.write("이것은  $\frac{1}{x}$ 입니다.")  
  
    st.write(dir(st))  
    st.help(st.write)  
  
if __name__ == '__main__':  
    main()
```



5

5

이것은 title입니다.

이것은 title입니다.

이것은 $\frac{1}{x}$ 입니다.

▶ [0 - 100]

▶ [100 - 182]

main() method streamlit.delta_generator.WriteMixin.write(*args: 'Any', unsafe_allow_html: 'bool' = False, **kwargs) -> 'None'

Write arguments to the app.

This is the Swiss Army knife of Streamlit commands: it does different things depending on what you throw at it. Unlike other Streamlit commands,

Widgets

```
#Button/Radio Button/Checkbox/Select
st.button('버튼 위젯')
```

```
name = "김길동"
if st.button("Submit"):
    st.write("이름: {}".format(name))
```

버튼 위젯

Submit

이름: 김길동

```
status = st.radio("선택", ("활성화", "비활성화"))
if status == "활성화":
    st.success("활성화 되었습니다.")
else:
    st.warning("비활성화 되었습니다.")
```

선택

☒ 활성화

☐ 비활성화

활성화 되었습니다.

```
# Slider
age = st.slider("나이", 1, 100, 50)
st.write("나이는 {}입니다.".format(age))
```

나이



나이는 37입니다.

```
if st.checkbox("show/hide"):
    st.text("Showing something")
```

☒ show/hide

Showing something

Text Input

```
# Text Input
name = st.text_input("이름을 입력하세요.")
st.write(name)

password = st.text_input("비밀번호를 입력하세요.", type="password")
st.write(password)
```

이름을 입력하세요.

김길동

김길동

비밀번호를 입력하세요.

.....



yeMryTDvb3kxvwR

Widgets: 입력 위젯 예시

```
# Text Area
message = st.text_area("메시지를 입력하세요.", height=100)
st.write(message)

# Numbers
number = st.number_input("숫자를 입력하세요.")
st.write(number)

# Date Input
appointment_date = st.date_input("약속 일자를 입력하세요.")
st.write(appointment_date)
```

메시지를 입력하세요.

안녕하세요. 좋은 아침입니다.

안녕하세요. 좋은 아침입니다.

숫자를 입력하세요.

300.00

300.0

약속 일자를 입력하세요.

2024/07/19

2024-07-19

< July 2024 >

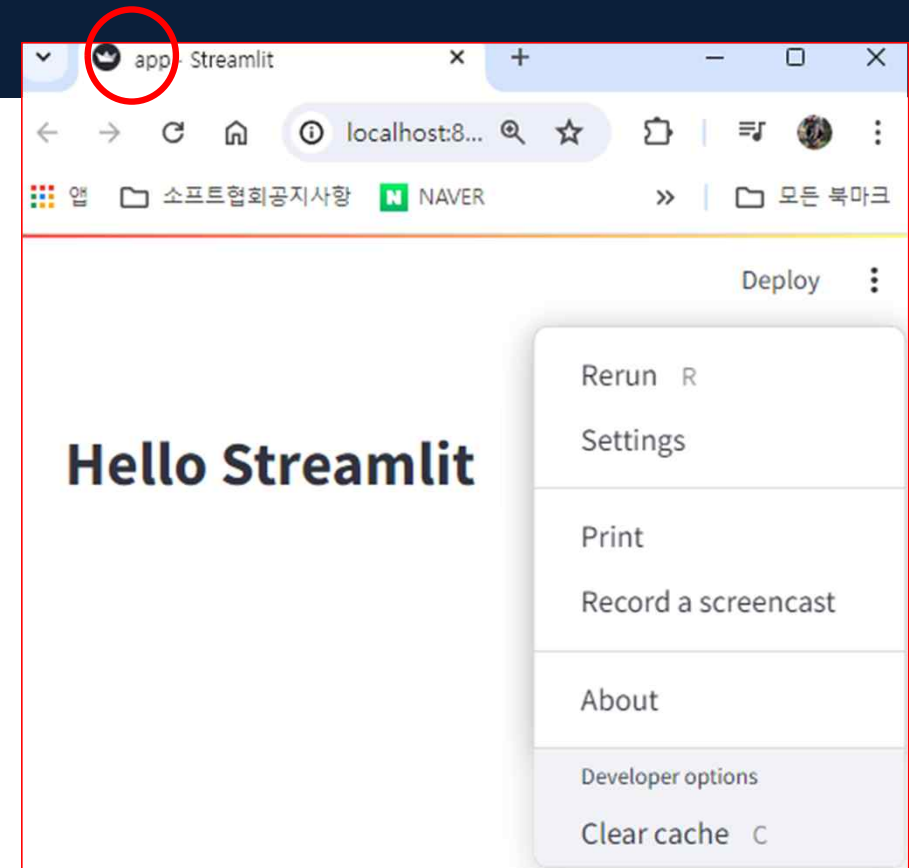
Su	Mo	Tu	We	Th	Fr	Sa
	1	2	3	4	5	6
7	8	9	10	11	12	13
14	15	16	17	18	19	20
21	22	23	24	25	26	27
28	29	30	31			

2024/07/19

Streamlit 실행

- streamlit run app.py

```
app.py > ...  
1 import streamlit as st  
2  
3 def main():  
4     st.title("Hello Streamlit")  
5  
6 if __name__ == '__main__':  
7     main()
```



```
(base) C:\Users\trimu\OneDrive\streamlit\LearnStreamlit-Udemy\Module04>streamlit run app.py
```

You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501

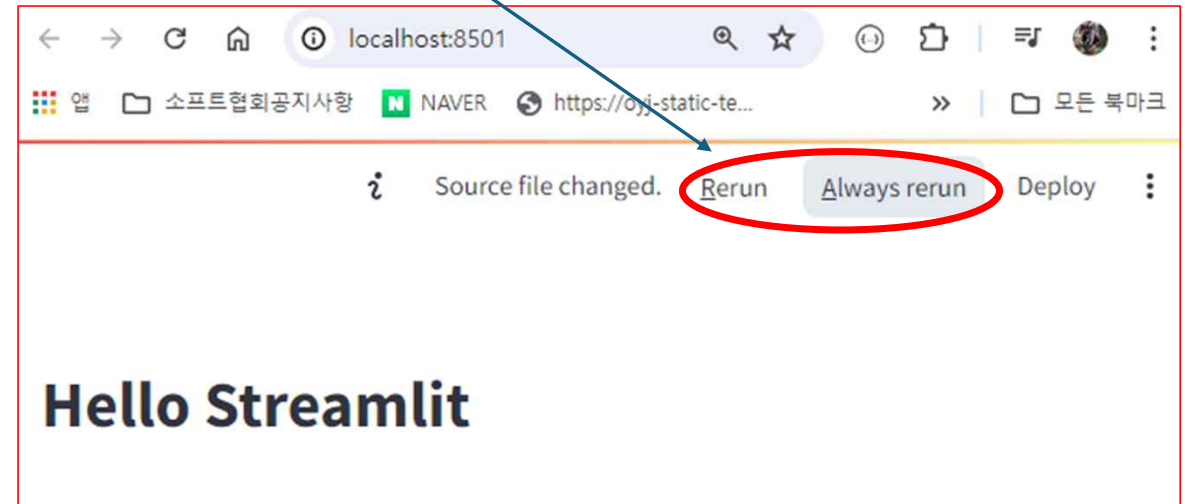
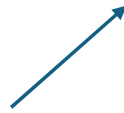
Network URL: http://172.30.1.40:8501

실시간 application update

```
app.py > ...  
1 import streamlit as st  
2  
3 def main():  
4     st.title("Hello Streamlit")  
5  
6 if __name__ == '__main__':  
7     main()
```



```
app.py > ...  
1 import streamlit as st  
2  
3 def main():  
4     st.title("반가워요 Streamlit")  
5  
6 if __name__ == '__main__':  
7     main()
```



반가워요 Streamlit

App 작성 순서

1. title 추가
2. Sidebar 추가
3. 입력 area widget 추가
4. Button widget 추가
5. OpenAI API 함수 추가
6. Button widget click 과 함수 연결

- **Streamlit 채팅 UI 구성**
- **대화 레이아웃**
 - 사용자 메시지: 오른쪽 열 / 어시스턴트 메시지: 왼쪽 열
 - 대화 기록은 세션에 유지
- **에이전트 동작**
 - 날씨: get_weather(위도, 경도) — open-meteo API
 - 웹 검색: WebSearchTool — 최신 정보 검색
 - RAG: search_knowledge_base — FAQ/문서 유사도 검색 후 답변

순차 실행 패턴 (Sequential)

에이전트 → 에이전트 체인



병렬 실행 패턴 (Parallel)

Parallel

병렬 실행

`asyncio.gather` 로 동시 실행 후 결과 통합

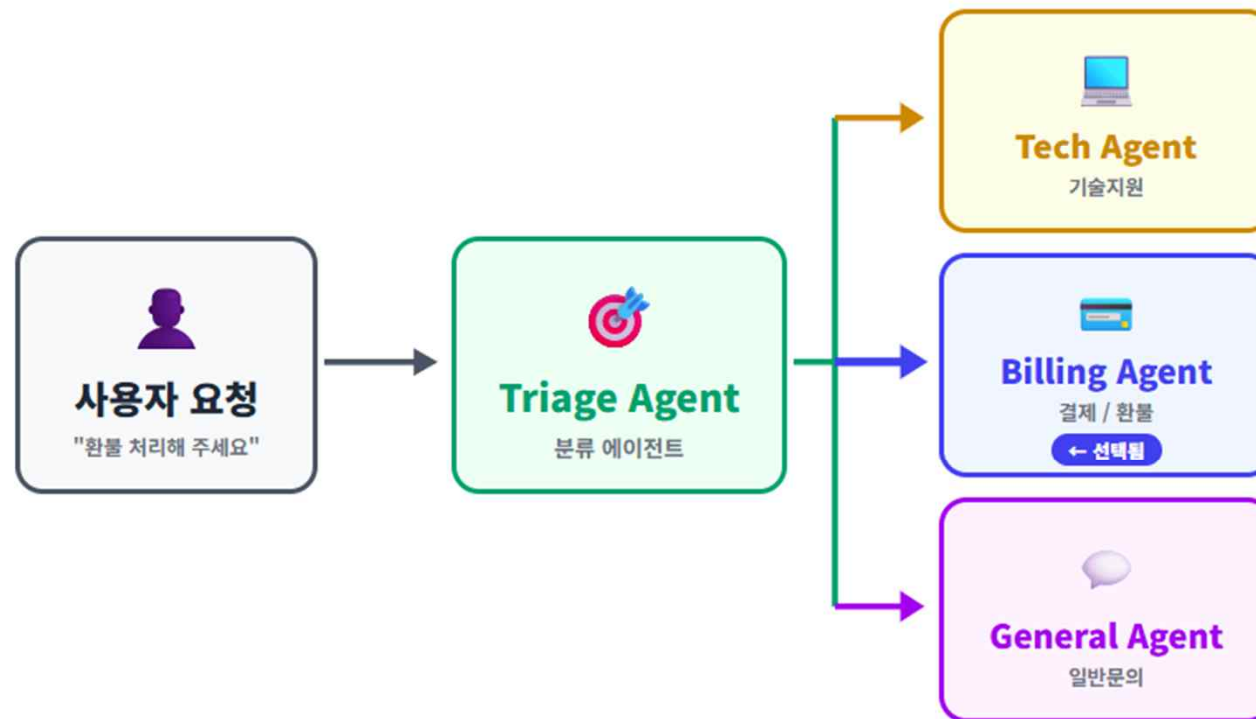


분류 + 라우팅 패턴

Routing

분류 + 라우팅

Triage 에이전트가 입력 분류 → 전문 에이전트로 자동 핸드오프



12. Multi-Agent Orchestration (코드 기반 멀티에이전트 오케스트레이션)

04_Handoffs에서는 LLM이 자율적으로 핸드오프를 결정하는 LLM 기반 오케스트레이션을 배웠습니다. 이번에는 코드로 직접 에이전트 흐름을 제어하는 패턴을 학습합니다.

LLM 기반 vs 코드 기반 오케스트레이션

구분	LLM 기반 (Handoffs)	코드 기반
제어 주체	LLM이 자율 판단	개발자가 코드로 제어
예측 가능성	낮음 (LLM 판단에 의존)	높음 (결정적 흐름)
유연성	높음	보통
비용/속도	변동적	예측 가능
적합한 경우	복잡한 대화, 분류가 어려운 경우	파이프라인, 배치 처리, 정해진 워크플로우

주요 패턴

1. 순차 체이닝 (Sequential Chaining): $A \rightarrow B \rightarrow C$
2. 병렬 실행 (Parallelization): A, B, C 동시 실행
3. 분류 → 라우팅 (Classification + Routing): 분류 후 전문 에이전트에 전달

- Streamlit 채팅 UI 구성

- 에이전트 동작

- 1) 순차 체이닝: 작성 → 편집 → 번역
- 2) 병렬 실행: 감성 분석 + 키워드 추출 + 요약 동시 실행
- 3) 분류 → 라우팅: 고객 문의를 분류 후 전문 에이전트에 전달
- 4) 종합 패턴: 병렬 분석 → 종합 보고서